

Allergen & Food Safety Hazard Compass

Detecting health hazards hidden in Yelp restaurant reviews

Iris Kronfeld · Tal Edrehy

Text Mining 2026 — Prof. Shay Palachy Affek · TA: Tal Cordova

The one-sentence version

We built a hazard detector — then spent the project proving our own ground truth wrong, **twice**, before trusting a single number.

The problem: aggregate rating masking

- A 4.5★ restaurant can hide a systemic kitchen failure. 98% of reviewers love the ambiance; 2% end up in the ER. The average hides the tail — and the tail is the safety signal.
- For allergy and celiac diners, a star average makes every meal an invisible gamble.

The cost asymmetry drives every design choice downstream

	Cost	Why
False negative — missed hazard	\$5,000	ER visit, litigation, an unwarned diner
False positive — false alarm	\$50	~10 minutes of human triage

100 : 1 — so we optimise **recall-weighted** metrics (F2), deploy a **lowered threshold** (0.20), and train a **cost-sensitive loss**.

Two research questions: ① Can we *detect* hazard reviews from raw text?
② Can we *discover hazard types* without labels?

Evidence: `COST_FALSE_NEGATIVE / COST_FALSE_POSITIVE` in `analysis/evaluation_pipeline.py` — the 100:1 ratio reproduces every cost figure in results/

Data & the weak label we started with

- Yelp Academic Dataset → food/restaurant businesses only, reviews 5–800 words
- Streamed and keyword-filtered until **1,500 flagged + 6,000 benign = 7,500 rows**
- Hazard reviews are rare enough that reaching 1,500 required **streaming and keyword-filtering the corpus** — this is *why* a funnel exists at all, and why every gold rate in this deck is a *funnel* rate rather than a population rate

The heuristic label

`is_hazard = 1` $\Leftrightarrow$ safety keyword (allergy, celiac, contamination, food poisoning, ...)

AND stars ≤ 3

- Split 64 / 16 / 20 → train 4,800 · **validation 1,200** · test 1,500
- Checkpoint selection and the hyperparameter objective see **only validation**, so test-split numbers are out-of-selection
- Metadata features engineered, then **deliberately withheld** from the transformer

Closing beat: *a rule-based label is a hypothesis, not ground truth — so we tested it.*

Act I — We audited our own label

An independent LLM judge re-labelled 1,500 rows blind, with no access to the rule.

	LLM: hazard	LLM: benign	
Rule: hazard	549	201	750
Rule: benign	12	738	750
	561	939	1,500

- Agreement 85.8% — but the disagreement is almost entirely one-sided
- **201 / 750 = 27% of the rule's positives are benign**; only 12 hazards missed
- Heuristic precision 73.2%, recall 97.9% — and that recall is circular: it is measured *inside the rule's own keyword filter*. Nothing outside the funnel can be counted.

Evidence: results/label_quality.json · labeling/gold_dataset.csv (1,500 rows) · labeling/create_gold_dataset.py

Why the rule over-flags — a failure taxonomy

We bucketed all 201 false positives by cause. This is a rule-based taxonomy, not hand-waving.

Failure mode	Share	Real example
Illness mentioned, not caused here	45.8%	"haven't gotten sick from them in 10 years"
Residual / unexplained	21.4%	—
Negated hazard	14.9%	"nothing here to trigger my allergy"
Neutral allergen mention	11.4%	"great gluten-free options"
Secondhand · hypothetical · unpleasant-not-unsafe · other	6.5%	"my friend said she felt off"

The mechanism, in one line

Co-occurrence is not causation.

The rule matches a keyword and a low star rating. It cannot represent **negation**, **attribution**, or **agency** — the exact three things bag-of-words methods cannot represent either. Remember this when we get to the model's own errors.

Act II — Our *evaluation* set was contaminated too

We built a gold set. Then we checked it against the training split.

- **1,334 / 1,500 rows (89%) of the first gold set sat in the training split.**
Evaluating on it would have measured **memorisation**, and it would have looked *excellent*.
- We caught this ourselves, before reporting a single model number.

The fix — a genuinely fresh holdout

- Rebuilt a candidate pool **straight from raw Yelp**, identical normalisation
(`labeling/build_holdout_pool.py`)
- **Verified 0 / 772 text overlap** with all training data — the check re-runs in the
`verify_setup.py` preflight, so it cannot silently rot
- LLM-labelled: **772 rows · 355 hazards · 46.0%**

⚠ **Say the caveat before anyone asks:** 46.0% is a **funnel** rate — 100% of these rows passed the keyword screen. Population prevalence is ~2–5%. Every gold number in this deck is "performance *within* the screened funnel".

Technique 1 — Fine-tuned DeBERTa-v3 + a cost-sensitive loss *family*

`microsoft/deberta-v3-base` , text only. Metadata withheld on purpose: `stars` , `medical_lexicon_density` and `negation_window_flag` all derive from the labelling rule, so feeding them back would be **direct label leakage**.

We did not build one loss — we built three, so they could be compared

Variant	Formulation	Honest description
<code>pos_weight</code>	<code>BCEWithLogits(pos_weight=w)</code>	This is standard class weighting. We say so.
<code>focal_asymmetric</code>	<code>1 + (w-1)(1-p)^γ</code> , $\gamma=2$	Penalty scales with the error , decays as confidence rises
<code>fn_gated</code>	penalty only where <code>p < τ</code>	Implemented + unit-tested, never run — we say that too

- Deployed: `focal_asymmetric @ w=50` , threshold **0.20** — lowered from 0.50 *because of* the 100:1 asymmetry, but **not read off it**. ⚠ Taken literally, a 100:1 ratio puts the cost-minimising threshold at **0.01** — and that is exactly what our pipeline measured on both evaluation sets. A 0.01 threshold means *flag essentially everything*, which is useless.
We reject our own cost model's optimum — and that is a second, independent reason cost is not our selection metric.

- **9 unit tests**, including a regression guard asserting an all-positive model is caught by

Act III — The metric that selects a broken model

Our hyperparameter sweep produced a **degenerate model**, and it is on disk.

Optuna trial 1 — lr = 3.80e-05, batch = 4, loss held at focal_asymmetric@50 :

	flag rate	recall	precision	PR-AUC
Trial 0 (lr 1.83e-05, bs 16)	0.196	0.913	0.932	0.9877
Trial 2 (lr 1.29e-05, bs 8)	0.203	0.938	0.926	0.9876
Trial 1 (lr 3.80e-05, bs 4)	1.000	1.000	0.200	0.196

Precision 0.200 is the base rate. It flagged every single review, at **every epoch** — its F2 sat at exactly **0.5556**, the analytic value of an all-positive answer at a 20% base rate.

Which objective would have picked it?

recall →	pr_auc →	f2 / f1 →
trial 1. The broken one.	trial 0; trial 1 ranks last (0.196 vs 0.988)	trial 2; also rejects trial 1

- ⚠ Be precise about what did and did not recover. The *hard predictions* never recovered — all-positive at every epoch. The *ranking* partially did: trial 1's best-epoch PR-AUC reached 0.643, against 0.988 for the two stable trials. Still last, either way.

THE payoff — same models, two ground truths

Gold holdout = 772 rows that never touched training, selection, or tuning.

Model	PR-AUC (heuristic)	PR-AUC (gold)	Δ
Baseline: TF-IDF + XGBoost + metadata	0.9785	0.7279	-0.2506
DeBERTa-v3 (deployed, th=0.20) — text only	0.9874	0.8045	-0.1830

Note the handicap runs *against* us: the baseline gets TF-IDF **plus all 7 tabular features**; the transformer sees raw text alone. The advantaged model is the one that loses.

Read the two columns against each other — this is the whole slide

- On **our own label**, the bag-of-words baseline is within **0.009** of a transformer. It looks like the transformer was a waste of a GPU.
- On **honest ground truth**, DeBERTa leads by **0.077** — an 8× wider gap.
- The baseline **falls further** (-0.251 vs -0.183, 1.37×) because **TF-IDF partially re-learns the keyword rule**. It was never doing the task; it was reverse-engineering our regex.

The flawed label did not merely inflate scores — it inverted the comparison between techniques.

Why we needed a gold set at all: the heuristic metric is *saturated*

We trained 8 configurations — `{pos_weight, focal_asymmetric}` × `{w = 1, 5, 15, 50}` — under one identical protocol (3 epochs, 900 optimiser steps each, verified from the log).

Ground truth	PR-AUC range across 8 configs	Spread
Heuristic label (validation)	0.9864 – 0.9894	0.0030
Gold label (fresh holdout)	0.7219 – 0.8211	0.0992

33× wider on real data.

The sharper statistic — rank correlation between the two ground truths

Spearman $\rho = -0.24$ ($p = 0.57$; Pearson $r = -0.36$)

The spread ratio is the claim that carries: our label had **almost no resolution** — 0.0030 of spread to distinguish eight models.

The rank correlation adds that there is **no detectable relationship** between the two orderings, and if anything it points mildly the wrong way. ⚠️ **We state the limit ourselves:** at $n=8$, $p=0.57$, this shows we *cannot demonstrate* the validation ranking carries information — it is not

What the custom loss actually buys — shape, not a ranking

Gold PR-AUC as the false-negative penalty grows:

Weight	pos_weight	focal_asymmetric
1	0.8096	0.7913
5	0.7956	0.7781
15	0.7905	0.8211
50	0.7219	0.8021
Range	0.0877	0.0430

- `pos_weight` declines **monotonically** across all four weights. Under pure noise, drawing an exactly monotone ordering of 4 points in the direction theory predicts has probability $1/24 \approx 4\%$ (one-sided, pre-specified). A uniform penalty on *every* positive, cranked to 50, over-penalises and measurably damages out-of-sample ranking.
- `focal_asymmetric` is **non-monotone** in a band **half as wide** — because its penalty decays as $(1-p)^\gamma$, so once the model is confident, the weight stops mattering. **That is what it was designed to do.**

The custom loss buys insensitivity to a hyperparameter, not peak

Technique 2 — Topic modelling for hazard-type discovery

LDA + NMF over the 1,500 flagged reviews. Shared vocabulary, domain stopwords, $K \in \{2,3,4,5,6,8,10\}$.

Selecting K honestly required rejecting our first criterion

- **Coherence alone is untrustworthy here.** NMF's highest NPMI is at $K=2$ — a degenerate fit where one topic holds **95% of documents**. NMF $K \in \{2,3,4,5\}$ *all* have a topic holding $>60\%$.
- So: **exclude degenerate fits first**, then report both the coherence-selected and the externally-validated K when they disagree (NMF: coherence $K=8$, validation $K=6$).
- **External validation is the stable criterion:** `nmi_above_null` picks **LDA $K=4$** and **NMF $K=6$** — the reported models.

⚠ Disclose in one breath: K was selected using the LLM hazard types (**mild circularity** — we report it), and $\sim 27\%$ of the fit corpus is benign **per our own Act I audit**.

Evidence: src/topic_model.py · results/topic_model_sweep.csv · results/topic_model_selection.png

What topic modelling can and cannot find

✓ It finds the rare, lexically-distinct hazard

NMF K=6, topic 1: *gluten · gluten free · celiac · cross contamination · gf · celiac disease*

- Lift 5.28 for `allergic_reaction` — the strongest association in the study
- Per-topic NPMI +0.43 — by far the most coherent topic found (the worst scores -0.22)
- ⚠ Own the limit: that topic is 5.28× *enriched* for allergen hazards but **captures only 7 of the 38** such documents. High precision on a narrow slice, not broad coverage.

✗ It cannot subdivide the common, lexically-diffuse one

- `food_poisoning` is $376 / 545 = 69\%$ of the validated set and stays one undifferentiated mass
- Overall purity 0.697 (NMF K=6) vs a majority-class baseline of 0.690 — essentially no gain
- NMI 0.08–0.12 against a shuffle null of 0.005–0.014: real, but weak

Topic models find the hazard with its own vocabulary, and fail on the hazard that shares everyone else's.

That is a property of the technique — distributional co-occurrence needs distinctive words

Where the *model* still fails — and it inherits its teacher's blind spots

DeBERTa on the gold holdout: 197 false positives, 39 false negatives. Same taxonomy as the label-audit slide.

False positive mode	Share
Illness mentioned, not caused here	32.5%
Neutral allergen mention	32.0%
Generic complaint (11.2%) + residual (11.2%)	22.3%
Negated hazard	8.6%
Secondhand · hyperbole · unpleasant-not-unsafe · other	4.6%

65% of the model's false alarms sit in the labelling rule's own top two failure modes.

The transformer did not invent new errors. It learned our rule's blind spots — the strongest analytical result in the project, because it traces a model failure back to a *label* pathology.

We also audited our own error taxonomy — and found the defect was ours

- 23 of 39 FNs first landed in `unexplained_fn` (59%). Cause: every FN rule was conditioned on

Limitations we report rather than hide

#	Limitation	Our disclosure
1	Run-to-run noise up to 0.054 gold PR-AUC, single seed (42)	Report weight-response <i>shape</i> , not a ranking; multi-seed study out of budget (~1 GPU-hr/cell)
2	Deployed config is not the grid leader — 3rd of 8; 0.019 behind <code>focal@15</code> , 0.008 behind <code>pos_weight@1</code>	Weight was fixed <i>before</i> the grid ran; both margins are inside our own noise floor, so we report the grid rather than retrofit the deployment
3	Three gold metrics, three winners. Deployed config is 3/8 on PR-AUC but 8/8 on F2 and 7/8 on cost	F2 and cost are computed at a fixed <code>th=0.20</code> and reward over-flagging (<code>focal@5</code> flags 75% of the holdout). We select on the threshold-free metric.
4	th=0.20 does not transfer . Gold flag rate 0.659–0.750 vs a 46.0% base rate; gold precision 0.57–0.63	Threshold was tuned against the heuristic label. Same models flag 18.6–20.8% on validation. One more instance of the central thesis — re-tune per distribution.
5	Hyperparameters : 3-trial sweep only; reported system trained at <code>lr 1.814e-05</code> , sweep best <code>1.835e-05</code>	A 1.1% difference, far inside the 0.054 noise floor. <code>analysis.py</code> now warns on the mismatch.
6	LDA irreproducible across machines; 46.0% is a funnel rate ; <code>fn_gated</code> never run	All three stated on their own slides, unprompted

Conclusions

The methodology arc — this is what we are actually submitting

built a heuristic label → **audited it** (85.8% agreement, 27% one-sided over-flagging) → explained *why* it errs, bucket by bucket → **caught our gold set 89% contaminated** → built a verified zero-overlap holdout (772 rows) → trained 8 loss configurations under one protocol → **found we could not tell them apart on our own label at all** (spread 0.0030 vs 0.0992, $p = -0.24$) → **measured our own run-to-run noise** and let it forbid ranking → reported *shape*, not leaderboard → showed the model's residual false alarms **are the label's own inherited blind spots**

Answers to the two research questions

- ① **Detection: yes, and the honest number is lower than the flattering one.** Gold PR-AUC **0.804** vs 0.987 against our own label. Recall 0.890 at 100:1 cost, **\$204,850** risk cost against the baseline's **\$585,300**.
- ② **Discovery: partially, and the failure is informative.** Allergen/celiac hazards separate cleanly and unsupervised (lift **5.28**, NPMI **+0.43**); the food-poisoning mass (69%) does not.

Business recommendation

Deploy as a screening funnel, not a verdict — threshold informed by the 100:1 asymmetry but

Thank you

Questions?

Every number in this deck traces to a committed artifact in `results/`.

Repo: code · notebooks · 9 unit tests · 8-cell loss grid · 3-trial sweep · two gold sets · full error taxonomies

APPENDIX (not presented) — Q&A back-pocket

1. "Is your asymmetric loss just class weighting?"

The `pos_weight` variant is, exactly, and we say so on the slide. `focal_asymmetric` is error-dependent. The grid compares both across 4 weights (8 cells). `fn_gated` is implemented and unit-tested but **never run** — we say that rather than imply coverage.

2. "46% hazard rate — really?"

Funnel rate; 100% of the holdout passed the keyword screen. The screen is two-tier, and the tiers behave very differently: **strong-term tier (n=378) 71.2% hazard, weak-term tier + stars $\leq$ 3 (n=394) 21.8%**. So the 46.0% is a mixture of a high-precision and a low-precision funnel, not a uniform rate. Population prevalence ~2–5%.

3. "Why w=50 when your cost ratio implies 100:1?"

The weight was fixed before the grid ran. The grid then ranked two configs ahead of it on gold — `focal@15` by **0.019** and `pos_weight@1` by **0.008** — both **far smaller than our measured run-to-run discrepancy (up to 0.054)**.

So we report the grid rather than retrofit the deployment to noise. Re-running `analysis.py` to chase 0.019 would cost 2–3 h and regenerate every artifact to buy a difference the data cannot resolve.

4. "Did you use the test set for selection?"

APPENDIX (not presented) — Figure export checklist

- [] **Slide 5** — 2×2 confusion table, heuristic vs LLM (549 / 201 / 12 / 738)
- [] **Slide 6** — horizontal bar chart of the 201 FP buckets
- [] **Slide 7** — contamination diagram: 1,334/1,500 overlap → 0/772 clean
- [] **Slide 9** — 3-trial table with trial 1 highlighted red (flag rate 1.000)
- [] **Slide 10** — grouped bars: heuristic vs gold PR-AUC, baseline vs DeBERTa (the payoff figure)
- [] **Slide 11** — dot plot: 8 configs on validation (clustered) vs gold (spread) — shows 33× visually
- [] **Slide 12** — weight-response curve, `pos_weight` monotone vs `focal` flat, with a ± 0.054 noise band
- [] **Slide 14** — lift table heat-strip (5.28 highlighted)
- [] **Slide 15** — FP bucket comparison: rule vs model, side by side (the "inherited blind spots" figure)
- [] Reuse existing: `results/pr_curve_gold_*.png` , `results/cost_curve_gold_*.png` ,
`results/topic_model_selection.png`

Rehearsal notes

- Target 14:00 to leave buffer. Slides 5–7 (the two audits) and 9–12 (metric, payoff, saturation, weight response) are the differentiators — do not rush them